

RAG Basics

Retrieval-Augmented Generation, or RAG, is a method that combines information retrieval with language model generation. Instead of relying only on knowledge learned during training, a RAG system searches an external collection for information relevant to a question. The basic workflow is query, retrieval, context, and generation. A user submits a query, the system retrieves useful passages, and those passages become context supplied to the language model.

The language model then generates an answer using the question and the retrieved context. For example, a question about a company policy can be answered using passages from its policy documents. Updating the document collection can make new information available without retraining the model. However, retrieval can miss relevant evidence, and generation can still introduce mistakes, so answers should be checked against their supporting passages.

Vector Stores

A vector store saves embeddings and supports searching them efficiently. An embedding is a numerical representation of text that captures aspects of its meaning. During indexing, a RAG system computes an embedding for each document chunk and stores it with the chunk text or a reference to that text. At query time, the system embeds the question using a compatible embedding model and compares it with the stored vectors.

This process enables semantic search, which can find related ideas even when the question and document use different words. Top-K retrieval returns the K highest ranked matches as candidate context. Document embeddings should be computed during indexing and reused, because recomputing every document embedding for every query wastes time and processing resources. Changed or new chunks need indexing updates; switching embedding models generally requires rebuilding the stored embeddings.

Chunking

Chunking divides a long document into smaller passages that can be indexed and retrieved separately. It matters because a question usually concerns only part of a document, and language models have limited context capacity. Retrieving focused chunks helps the system provide relevant evidence without filling the prompt with unrelated material. Good chunk boundaries preserve complete sentences and, when possible, keep closely related ideas together.

Chunk size controls how much text each passage contains, often measured in tokens. Very large chunks can dilute relevant details and consume too much context space. Very small chunks can lose definitions, explanations, or relationships needed to understand a statement. Chunk overlap repeats some text between neighboring passages to preserve context across boundaries, but excessive overlap increases storage and duplication. The best settings depend on document structure and should be tested with representative questions.

Metadata and Citations

Metadata describes a document chunk and helps a RAG system organize, filter, and trace retrieved information. Useful fields include the source file, page number, and section title. For example, a chunk may record source file `knowledge.pdf`, page number 4, and section title `Metadata and Citations`. Storing these fields alongside the text allows the system to retrieve passages from a selected document or page and maintain their original locations.

Citations connect claims in a generated answer to supporting passages. A citation containing the source file and page number tells users which document and page to inspect. This makes answers easier to verify, helps users notice unsupported claims, and supports debugging when retrieval selects the wrong evidence. Citations improve transparency, but they do not guarantee correctness: the cited passage must actually support the claim, and the source itself must be reliable.